



FitStudent

תזונה ואימונים לסטודנטים · קבוצה 42 · יסודות מערכות מידע · אוניברסיטת בן-גוריון

יוני 2026

גרסה alpha-1.0.0

תיעוד שלב Alpha 1

תוכן עניינים

1. תקציר הפרויקט
2. ארכיטקטורה כללית
3. מחסנית הטכנולוגיות (Tech Stack)
4. האינטגרציות: מה, איך ולמה
5. מסד הנתונים ואבטחה (RLS)
6. זרימת המשתמש
7. שלבי הפיתוח (Step by Step)
8. מה הלאה - < שלב 2

1 תקציר הפרויקט

אפליקציית ווב מובייל-תחילה (mobile-first) לניהול תזונה ואימונים, מותאמת לסטודנטים.

FitStudent היא אפליקציית Single Page Application (SPA) בעברית מלאה (RTL) המאפשרת לכל סטודנט:

- להירשם, להתחבר, ולמלא שאלון היכרות (Onboarding) רב-שלבי.
- לקבל יעדים תזונתיים אישיים המחושבים בנוסחת Mifflin-St Jeor (BMR, TDEE), קלוריות ומאקרו-נוטריאנטים).
- לעקוב אחר תזונה יומית: יומן, צילום ארוחה, הוספה ידנית, בונה ארוחות ומתכונים.

- לקבל ולערוך תוכנית אימונים שבועית (חדר כושר / יוגה / פילאטיס / קרוספיט).

- לנהל פרופיל, יעדים, גרף משקל, מעקב מים, ומצב יום/לילה.

הנתונים נשמרים בענן ומשויכים למשתמש כל משתמש רואה אך ורק את המידע שלו. האפליקציה פרוסה (deployed) וזמינה בקישור ציבורי.

<https://fit-student.vercel.app>

קישור חי

github.com/Eithan200/FitStudent

קוד מקור

שלב 1 (Alpha) הושלם ונבדק על-ידי הצוות ☒

סטטוס

2 ארכיטקטורה כללית

ארכיטקטורת BaaS → Client: דפדפן מול שירות ענן מנוהל, ללא שרת ביניים משלנו.

המבנה מורכב משתי שכבות בלבד, מה שמתאים לפרויקט סטודנטיאלי פשוט, מהיר לפיתוח, וזול (חינמי) לאחסון:

- **שכבת לקוח (Frontend):** אפליקציית React שרצה כולה בדפדפן, נבנית עם Vite ומאוחסנת כקבצים סטטיים ב-Vercel.

- **שכבת ענן (Backend-as-a-Service):** Supabase מספקת מסד נתונים PostgreSQL, מערכת אימות (Auth), ואבטחת-שורה (Row Level Security) ללא צורך לכתוב שרת.

זרימת הנתונים:

[React SPA • Browser] ⇔ [@supabase/supabase-js] ⇔ [Supabase: Auth + PostgreSQL + RLS]

בשלב 2 תתווסף שכבת אינטגרציית-AI (ראה [פרק 8](#)), שתתחבר דרך שכבת-ביניים (serverless) כדי לשמור על מפתחות ה-API מוסתרים.

3 מחסנית הטכנולוגיות

סיכום מהיר של כל הכלים והגרסאות בהן השתמשנו בשלב 1.

קטגוריה	טכנולוגיה	גרסה	תפקיד
Framework	<code>react</code> / <code>react-dom</code>	18.3	בניית ממשק רכיבי
Build tool	<code>vite</code>	5.4	שרת פיתוח + bundling
Routing	<code>react-router-dom</code>	6.26	ניווט בין מסכים
Styling	<code>tailwindcss</code>	3.4	עיצוב מבוסס utility
Backend (BaaS)	<code>@supabase/supabase-js</code>	2.45	אימות + מסד נתונים
Charts	<code>recharts</code>	2.12	גרפים
Icons	<code>lucide-react</code>	0.441	אייקונים
Hosting / CI	Vercel + GitHub		אחסון ופריסה אוטומטית

4 האינטגרציות מה, איך ולמה

לכל אינטגרציה: **מה** היא, **איך** השתמשנו בה בפרויקט, ו**למה** בחרנו בה.

React + Vite

react 18.3 · vite 5.4

- מה

React היא ספריית JavaScript לבניית ממשקי משתמש מרכיבים (components). Vite הוא כלי הבנייה ושרת הפיתוח.
- איך

כל מסך באפליקציה הוא רכיב (React (Home, Nutrition, Workouts, Profile, Onboarding) השתמשנו ב- `useState`, `useEffect`, `useContext` (Hooks) לניהול מצב. Vite מספק טעינה-חמה (HMR) בפיתוח ובונה קבצים מוקטנים (minified) לפריסה.
- למה

React הוא הסטנדרט התעשייתי לממשקים אינטראקטיביים, עם קהילה ענקית ותיעוד נרחב. Vite מהיר משמעותית מכלים ישנים (כמו Create React App) וקל להגדרה.

React Router

react-router-dom 6.26



מה

ספריית ניתוב (routing) המאפשרת ניווט בין "עמודים" שונים ב-SPA ללא רענון הדף.

איך

הגדרנו נתיבים (/home, /nutrition, /workouts, /profile, /onboarding), יצרנו רכיב Protected, המגן על נתיבים מי שאינו מחובר מופנה להתחברות, ומי שאין לו פרופיל מופנה ל-Onboarding. לשוניות-המשנה (tabs) נשמרות ב-query string (למשל ?tab=diary).

למה

פתרון ניתוב בשל ותקני ל-React; ה-SPA נטען פעם אחת והמעבר בין מסכים מיידי חוויה דמוית-אפליקציה במובייל.

Tailwind CSS

tailwindcss 3.4



מה

מסגרת עיצוב מבוססת utility classes מעצבים ישירות ב-HTML בעזרת מחלקות קטנות.

איך

הגדרנו CSS Variables מותאמות לפלטת הצבעים של הפרויקט (ירוק-ליים על רקע כהה). מימשנו מצב יום/לילה דרך data-theme על שורש המסמך, כך שכל הצבעים מתחלפים אוטומטית. רכיבים חוזרים (כרטיסים, כפתורים, ציפים) הוגדרו כ-component classes.

למה

פיתוח עיצוב מהיר ועקבי ללא קבצי CSS מנופחים; קל לתחזק ערכת-נושא אחת ולהחיל אותה על כל האפליקציה.

Supabase

@supabase/supabase-js 2.45 Auth · PostgreSQL · RLS



מה

Supabase היא פלטפורמת Backend-as-a-Service מבוססת קוד-פתוח: מסד PostgreSQL, מערכת אימות מובנית, ו-API אוטומטי.

איך

• אימות (Auth): הרשמה והתחברות באימייל+סיסמה דרך supabase.auth. מצב ההתחברות מנוהל ב-React Context שמאזין ל-onAuthStateChanged ומשחזר את ה-session ברענון.

- **מסד נתונים:** 6 טבלאות (ראה [פרק 5](#)). כל פעולת קריאה/כתיבה נעשית ישירות מהדפדפן דרך ה-client.
- **אבטחה:** Row Level Security מבטיח שכל משתמש ניגש רק לשורות שלו.

למה

מאפשרת אחסון מאובטח והתמדת-נתונים אמיתית ללא כתיבת שרת חוסכת זמן עצום בפרויקט סטודנטיאלי. השכבה החינמית מספיקה לחלוטין לקנה-המידה שלנו, והיא מבוססת על PostgreSQL הסטנדרטי.

Recharts

recharts 2.12



ספריית גרפים ל-React, בנויה מעל SVG.

מה

גרף עמודות (BarChart) במסך הבית להתקדמות קלורית שבועית מול קו-יעד, וגרף קו (LineChart) במסך הפרופיל למעקב משקל לאורך זמן.

איך

קל לשילוב עם React, רספונסיבי, וניתן לעיצוב מלא בצבעי האפליקציה מציג נתונים אמיתיים מהמסד בצורה ויזואלית וברורה.

למה

Lucide React

lucide-react 0.441



ספריית אייקונים בקוד פתוח בפורמט וקטורי (SVG), כרכיבי React.

מה

בשורת הניווט התחתונה, בכפתורים ובכתורות (מצלמה, מקרר, שעון מנוחה, טיפת מים, שמש/ירח למצב תצוגה ועוד).

איך

אייקונים חדים בכל גודל, משקל קל, ושליטה מלאה בצבע וגודל שומרים על מראה אחיד ומקצועי.

למה

Vercel + GitHub

Hosting & Continuous Deployment



GitHub ניהול גרסאות הקוד. Vercel פלטפורמת אחסון ופריסה לאתרים סטטיים/Frontend.

מה

איך

הקוד מנוהל ב-Git ומסונכרן ל-Vercel. GitHub מחובר למאגר כל `git push` ל-`main` מפעיל בנייה ופריסה אוטומטית (CI/CD) לקישור החי. קובץ `vercel.json` מגדיר SPA rewrites כדי שהניתוב יעבוד.

למה

פריסה אוטומטית, חינוכית ומיידית; הצוות יכול לעדכן ולשתף את האפליקציה בקלות; ניהול גרסאות מאפשר שיתוף-פעולה ושחזור.

Supabase MCP



(Developer Tooling) כלי פיתוח

מה

שרת Model Context Protocol רשמי של Supabase מאפשר לכלי-פיתוח לנהל את המסד.

איך

השתמשנו בו בזמן הפיתוח כדי להריץ migrations (יצירת/עדכון טבלאות ועמודות) ולוודא שינויים ישירות מסביבת הפיתוח, מבלי להיכנס ידנית ללוח הבקרה של Supabase.

למה

מזרז ומדייק את עבודת מסד-הנתונים בזמן הפיתוח. **אינו** חלק מהאפליקציה הפרוסה כלי-עזר בלבד.

כלי דפדפן מובנים



Google Fonts · Web Audio API · localStorage

מה

יכולות מובנות בדפדפן ושירות גופנים, ללא ספריות נוספות.

איך

- **Google Fonts (Inter)**: הגופן הראשי, התומך בעברית ובאנגלית.
- **Web Audio API**: צפצוף קצר בסיום טיימר המנוחה באימון.
- **localStorage**: שמירת העדפת מצב יום/לילה בין כניסות.

למה

מוסיפים חוויה מלוטשת (טיפוגרפיה, צליל, זיכרון העדפות) בעלות אפס וללא תלות חיצונית.

שכבת AI Stubs להכנה לשלב 2



src/services/ai.js

מה

פונקציות "מדומות" (mock) המחזירות נתוני-דוגמה, אך עם החתימה (signature) המדויקת שתידרש בשלב 2.

בשלב 1 פיצ'רים כמו "צלם ארוחה", "מהמקרר לצלחת" ובניית תוכנית האימונים מחזירים נתוני-mock לאחר השהיה קצרה (לדמות זמן-תגובה). הקוד מסומן ב- `// PHASE 2` במקומות שבהם תיכנס קריאת ה-AI האמיתית.

מאפשר לבנות ולבדוק את כל ממשק-המשתמש כבר עכשיו, ולהחליף בשלב 2 רק את גוף הפונקציה מבלי לגעת בשאר האפליקציה (separation of concerns).

5 מסד הנתונים ואבטחה

6 טבלאות ב-PostgreSQL, כולן מוגנות ב-Row Level Security.

טבלה	תפקיד
users	פרופיל המשתמש: נתונים אישיים, מטרה, סוג אימון, יעדים מחושבים (BMR/TDEE/מאקרו), יעד מים.
food_log	יומן תזונה: כל ארוחה עם סוג, פריטים, קלוריות ומאקרו.
workout_log	היסטוריית אימונים שבוצעו: סוג, רמה/סגנון, תרגילים, משך, קלוריות שנשרפו.
workout_plan	תוכנית האימונים השבועית (לפי יום בשבוע), ניתנת לעריכה פר-יום.
weight_log	מעקב משקל לאורך זמן (לגרף).
recipes_cache	מתכונים שמורים.

Row Level Security (RLS) אבטחת שורה: מנגנון של PostgreSQL המוגדר על כל טבלה, וקובע ש- `auth.uid() = user_id`. כלומר, גם אם הקריאה למסד מגיעה ישירות מהדפדפן, המסד עצמו מחזיר למשתמש **אך ורק את השורות שלו**. כך שני משתמשים שונים לעולם לא יראו את נתוני זה של זה האבטחה נאכפת בשכבת המסד, לא בקוד הלקוח.

6 זרימת המשתמש

המסלול מהרשמה ועד שימוש יומיומי.

- **הרשמה / התחברות** אימייל וסיסמה. ברענון הדף המשתמש נשאר מחובר.
- **Onboarding (4 שלבים)** פרטים אישיים → נתוני גוף → מטרה, רמת פעילות וסוג אימון מועדף → העדפות תזונה. בסיום מחושבים היעדים ונבנית תוכנית אימונים שבועית.
- **מסך הבית** טבעות קלוריות ומאקרו, האימון הבא, מתכון מומלץ, גרף שבועי, מעקב מים, וטיפ יומי.
- **תזונה** יומן, הוספת ארוחה (צילום / ידני / בונה ארוחות / מתכון), מהמקרר לצלחת, ומתכונים שמורים.
- **אימונים** התוכנית שלי (עריכה פר-יום), אימון היום, הוספת אימון ידני, היסטוריה, וספריית תרגילים.
- **פרופיל** פרטים, יעדים וגרף משקל, והגדרות (מצב תצוגה, יעד מים, פרטיות, מחיקת חשבון).

7 שלבי הפיתוח (Step by Step)

האבולוציה של שלב 1 מגרסת ה-Alpha הראשונה ועד התיקונים הסופיים, כפי שמתועד בהיסטוריית ה-Git.

Phase 1 - Alpha

בסיס האפליקציה

הקמת ה-SPA ב-React + Vite, חיבור ל-Supabase (אימות והתמדה), 5 המסכים, Onboarding, סקמת המסד עם RLS, ועיצוב כהה-ליים. כל משתמש יכול להירשם, להשלים Onboarding, ולהגיע למסך הבית עם נתונים שנשמרים.

Phase 1.1

סוגי אימון, רישום ידני, ותיקונים

הוספת דיסציפלינות אימון (חדר כושר / יוגה / פילאטיס / קרוספיט), רישום ידני של ארוחות ואימונים, יעד מים מתכוונן. הסרת מצב הדמו, ותיקון באג שהפנה משתמש קיים שוב ל-Onboarding בכל התחברות.

Phase 1.2

מצב יום/לילה ותזונה עשירה יותר

מצב תצוגה בהיר/כהה מלא (עוקב אחר הגדרת הטלפון, עם החלפה ידנית), שבוע אימונים מעורב (סוג שונה לכל יום), שדרוג זרימות התזונה, והוספת "מתכונים מומלצים".

Phase 1.3

תוכניות ניתנות לעריכה, רמות, ובונה ארוחות

עורך אימון משותף לעריכת כל יום (עבר ועתיד), פילאטיס לפי רמת קושי ויוגה לפי סגנון, חישוב קלוריות שנשרפות (מבוסס MET), ובונה ארוחות מרובה-פריטים. הסרת תמחור מהמתכונים, והתאמת מתכונים לפי המרכיבים שיש בבית.

Phase 1.3.1 1.3.2

ליטושים אחרונים

עריכת "אימון היום" במקום, ניקוי טקסט, ואפשרות לבחור מבין כמה אימונים מומלצים בעת טעינת המלצה. **בנקודה זו שלב 1 הושלם, נבדק על-ידי הצוות, ועובד במלואו.**

מה הלאה שלב 2 8

שלב 1 הניח תשתית מלאה ובדוקה. **שלב 2** יחליף את שכבת ה-AI המדומה באינטגרציות אמיתיות (למשל ניתוח תמונות מזון ואימות נתונים תזונתיים), דרך שכבת-ביניים מאובטחת שתסתיר את מפתחות ה-API.

המדריך המפורט לשלב 2 יכתב בהמשך, בהתאם להגדרות שתספקו.